Ezra BCruz

Problem 1

Recall that the bottom-up dynamic Programming alg for finding the nth Fib. number required $\Theta(n)$ extra space. Modify the alg to develop a linear time bottom up DP approach w. $O(1)$ extra space.

Hint: in order to compute $F(i)$, we only need to know $F(i-1)$ and $F(i-2)$ and do not require the previous Fib nums. Use this to replace the aux array of size n with 2 aux variable

```
Fib [n]

    Prev_Fib_One = 1        // base case
    Prev_Fib_Two = 1        // BC

    if n ≤ 2
        return prev_Fib_One // can use either aux var since BC=1

    for i = 3 to n          // For loop to update our aux vars until nec.

        current_Fib = prev_Fib_One + prev_Fib_Two  // local var
        //update aux vars
            prev_Fib_One = prev_Fib_Two
            Prev_Fib_Two = current_Fib

    return (prev_Fib_One + prev_Fib_Two)
```

Analysis

$TC = \Theta(n)$

$SC = O(1)$ extra space b/c only aux vars are used

Eatin Berry

Problem 2

Part A

Q2)

Recall the rod cutting problem... consider a rod of length n inches and an array of prices p[1, ..., n], where p[i] denotes the price of any i inches long piece of rod. Suppose we have to pay a cost of $1 per cut. If we cut the rod into k pieces of lengths $n_1, n_2, \ldots, n_k$, we made k-1 cuts, so final selling price is $p[n_1] + \ldots + p[n_k] - (k-1)$

Let maxprice(n) denote max selling price

a) Find the rec that mprice satisfies..

```
memo[0....n] = [ ]   // size = n+1 b/c we need to account for 0
maxprice(n)
    if memo[n] is NOT empty   // already computed this val
        return memo[n]

    memo[n] = 0   // base case

    for j = 1 to n
                                  // prev value of mem.
        memo[n] = max (maxprice(n), p[j] + maxprice(n-j))

    return memo[n].
```

This alg finds the max Price(n) in terms of maxprice(n-1), maxprice(n-2), ... maxprice(0).

Part B) the Base Case is when n = 0, thus the value is also zero since you cannot sell/have a price for nothing (a rod of zero inches)

Ezra Cruz

Problem 2 Part C

Bottom up DP

```
maxPrice(n)
    memo[0...n] = [0, ..., 0]   //size = n+1, this covers BaseCase
    for i = 1 to n
        for j = 1 to i
            memo[i] = max(memo[i], P[j] + memo[i-j])   // takes O(1)
    return memo[n]
```

$\Theta(i)$

$\sum_{i=1}^{n} \Theta i$

Time Complexity

$$TC = \sum_{i=1}^{n} \Theta(i) = \sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \frac{n^2+n}{2} = \frac{1}{2}n^2 + \frac{1}{2}n = \Theta(n^2)$$

Ester Cruz

Problem 3

Part A) Find Values of Makechange(8) and makechange(9)

make change (8)

① 1 1 1 1 1 1 1 1 (8 pennies)

② 1 1 1 5 (3 pennies, 1 nickel)

④ 1 1 5 1

⑤ 1 5 1 1

⑥ 5 1 1 1

make Change (9)

① 1 1 1 1 1 1 1 1 1 (9 pennies)

② 1 1 1 1 5 (4 pennies, 1 nickel)

④ 1 1 1 5 1

⑤ 1 1 5 1 1

⑥ 1 5 1 1 1

⑦ 5 1 1 1 1

Part B)

```
//options: coin can be either penny, nickel, or dime

//base case

    if n=1
        return 1 // only a penny.

    makechange(n) = makechange(n-1) + makechange(n-5) +
                                      makechange(n-10)

    //see how many coins you'll need
```

TC = $\Theta(N)$

Part C) Pseudo Code : DP Alg

```
memo[1,...,n] = [0, ..., 0]
memo[1] = 1 //Base case

for i = 2 to n   //runs n times
    if (i-1 >=0)
        memo[i] = memo[i-1]; //penny
    if (i-5 >=0)
        memo[i] = memo[i-5] //nickel
    if (i-10 >=0)
        memo[i] = memo[i-10] //dime
```